

Web Scrapping - Wikipedia

Need requests to download the web page HTML, BeautifulSoup to parse and search through that HTML structure, and pandas to turn the final extracted data into a neat table.

```
In [52]: from bs4 import BeautifulSoup
import pandas as pd
import requests

print("Libraries imported successfully!")
```

Libraries imported successfully!

Step 2: Send an HTTP Request to the URL

Output: Status Code: **200** (A 200 code means the server **successfully responded** and the page is accessible).

Output: A **403 error** means Wikipedia's server recognized your script as an automated bot and blocked access.

Pass a User-Agent header that mimics a real web browser.

```
In [53]: url = "https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_populati

# Add a User-Agent header so Wikipedia thinks you are a human using a browser
headers = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    )
}

response = requests.get(url, headers=headers)

print("Status Code:", response.status_code)
```

Status Code: 200

Step 3: Parse the HTML with BeautifulSoup

Explanation: BeautifulSoup takes the raw text from the response and transforms it into a navigable tree structure so you can search for specific HTML tags.

```
In [54]: soup = BeautifulSoup(response.text, "html.parser")

print("Page Title:", soup.title.text)
```

Page Title: List of countries and dependencies by population - Wikipedia

```
In [55]: # Wikipedia tables usually share the 'wikitable' class name
table = soup.find("table", class_="wikitable")

print("Table Found:", table.name)
print("Classes:", table.get("class"))
```

Table Found: table

Classes: ['wikitable', 'sortable', 'mw-datatable', 'sort-under', 'static-row-numbers', 'sticky-header', 'col1left', 'col5left']

Explanation: Instead of scanning the whole page, .find() targets the first HTML

element that has the class wikitable, which is where Wikipedia stores its structured datasets.

Step 5: Extract Rows and Columns

```
In [56]: data = []
rows = table.find_all("tr")

for row in rows:
    # Find both table headers (th) and table cells (td)
    cols = row.find_all(["th", "td"])
    cols = [ele.text.strip() for ele in cols]
    if cols: # Ignore empty rows
        data.append(cols)

print(f"Total rows extracted: {len(data)}")
print("First row (Headers):", data[:1])
```

Total rows extracted: 241

First row (Headers): [['Location', 'Population', '% ofworld', 'Date', 'Source (official or fromthe United Nations)', 'Notes']]

Step 6: Convert into a Pandas DataFrame

```
In [57]: df = pd.DataFrame(data)
df
```

Out[57]:

	0	1	2	3	4	5
0	Location	Population	% ofworld	Date	Source (official or fromthe United Nations)	Notes
1	World	8,232,000,000	100%	13 Jun 2025	UN projection[1][3]	
2	India	1,429,404,000	17.3%	1 Jul 2026	Official projection[4]	[b]
3	China	1,404,890,000	17.0%	31 Dec 2025	Official estimate[5]	[c]
4	United States	341,784,857	4.1%	1 Jul 2025	Official estimate[6]	[d]
...	...	...	...	...	...	...
236	Christmas Island (Australia)	1,692	0%	1 Jan 2021	2021 Census[248]	
237	Niue (New Zealand)	1,681	0%	11 Nov 2022	2022 Census[249]	
238	Vatican City	882	0%	31 Dec 2024	Official figure[250]	[ah]
239	Cocos (Keeling) Islands (Australia)	593	0%	30 Jun 2020	2021 Census[251]	
240	Pitcairn Islands (UK)	35	0%	2023	Official estimate[252]	

241 rows × 6 columns

```
In [58]: # Use the first row as column headers, and the remaining rows as data
df = pd.DataFrame(data[2:], columns=data[0])
df
```

Out[58]:

	Location	Population	% ofworld	Date	Source (official or fromthe United Nations)	Notes
0	India	1,429,404,000	17.3%	1 Jul 2026	Official projection[4]	[b]
1	China	1,404,890,000	17.0%	31 Dec 2025	Official estimate[5]	[c]
2	United States	341,784,857	4.1%	1 Jul 2025	Official estimate[6]	[d]
3	Indonesia	288,315,089	3.5%	31 Dec 2025	National annual projection[7]	
4	Pakistan	241,499,431	2.9%	1 Mar 2023	2023 census result[8]	[e]
...	...	...	...	...	...	...
234	Christmas Island (Australia)	1,692	0%	1 Jan 2021	2021 Census[248]	
235	Niue (New Zealand)	1,681	0%	11 Nov 2022	2022 Census[249]	
236	Vatican City	882	0%	31 Dec 2024	Official figure[250]	[ah]
237	Cocos (Keeling) Islands (Australia)	593	0%	30 Jun 2020	2021 Census[251]	
238	Pitcairn Islands (UK)	35	0%	2023	Official estimate[252]	

239 rows × 6 columns

```
In [59]: # Keep columns from index 0 up to (but not including) index 4
df2 = df.iloc[:, 0:4]
df2
```

Out[59]:

	Location	Population	% ofworld	Date
0	India	1,429,404,000	17.3%	1 Jul 2026
1	China	1,404,890,000	17.0%	31 Dec 2025
2	United States	341,784,857	4.1%	1 Jul 2025
3	Indonesia	288,315,089	3.5%	31 Dec 2025
4	Pakistan	241,499,431	2.9%	1 Mar 2023
...	...	...	...	...
234	Christmas Island (Australia)	1,692	0%	1 Jan 2021
235	Niue (New Zealand)	1,681	0%	11 Nov 2022
236	Vatican City	882	0%	31 Dec 2024
237	Cocos (Keeling) Islands (Australia)	593	0%	30 Jun 2020
238	Pitcairn Islands (UK)	35	0%	2023

239 rows × 4 columns

Step 7: Code to Export to CSV or Excel

```
In [60]: # Save the scraped DataFrame as a CSV file
csv_filename = "Scrapped.csv"
df2.to_csv(csv_filename, index=False, encoding="utf-8")
print(f"Data successfully saved to {csv_filename}!")
```

Data successfully saved to Scrapped.csv!

```
In [64]: # Use ExcelWriter with mode='a' to open the existing file
# if_sheet_exists='replace' will update/overwrite just the "Books" sheet inside the
with pd.ExcelWriter(excel_filename, engine='openpyxl', mode='a', if_sheet_exists='r
    df.to_excel(writer, index=False, sheet_name="Population Data")

print(f"Data successfully updated inside existing file {excel_filename}!")
```

Data successfully updated inside existing file Scrapped.xlsx!

In []: